Chapter 5

Conjugate gradient.

Abstract:
This chapter develops the method of conjugate directions, the linear conjugate-gradient algorithm, its convergence behavior on SPD systems, preconditioning, and the basic nonlinear conjugate-gradient family.

It is useful to place conjugate gradients between the two methods we already know best. If the cost of one objective/gradient evaluation is $O(q)$, then a rough comparison is so gradient descent is cheap but may zig-zag, while Newton is geometrically better but

Method	Memory	Per-iteration compute
Gradient descent (GD)	$O(n)$	$O(q+n)$
Newton (dense)	$O(n^2)$	$O(qn+n^3)$

Table 5.1: Rough per-iteration memory and computational cost for the two benchmark methods from earlier chapters. Here n is the dimension and q is the cost of one objective/gradient evaluation. The Newton row assumes a dense Hessian and a direct solve via a matrix factorization.

much more expensive. Conjugate gradients are designed to keep the storage and per-step structure much closer to gradient descent while still using the quadratic geometry induced by Newton method.

5.1 The method of conjugate directions

We begin with the quadratic problem

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x} \to \min_{\mathbf{x}\in\mathbb{R}^n}, \qquad \mathbf{A} = \mathbf{A}^\top \succ 0. \tag{5.1}$$

Its gradient is

$$\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b},$$

so the optimality condition is the linear system

$$\mathbf{A}\mathbf{x} = \mathbf{b}.$$

This is the best starting point to see why CG works. Afterwards, we will consider more complex cases.

5.1.1 Exact line search on a quadratic

Suppose we move from $\mathbf{x}_k$ along a direction $\mathbf{d}_k$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k.$$

For the quadratic (5.1), exact line search is explicit and we don't need Armijo–Wolfe. Let

$$\varphi_k(\alpha) := f(\mathbf{x}_k + \alpha \mathbf{d}_k), \qquad \mathbf{g}_k := \nabla f(\mathbf{x}_k) = \mathbf{A}\mathbf{x}_k - \mathbf{b}.$$

Then

$$\varphi_k'(\alpha) = (\mathbf{g}_k + \alpha \mathbf{A}\mathbf{d}_k)^\top \mathbf{d}_k,$$

so the minimizing step is

$$\alpha_k = -\frac{\mathbf{g}_k^\top \mathbf{d}_k}{\mathbf{d}_k^\top \mathbf{A}\mathbf{d}_k}. \tag{5.2}$$

Because $\mathbf{A} \succ 0$, the denominator is positive whenever $\mathbf{d}_k \neq \mathbf{0}$. The only thing left for CG method is defining directions $\mathbf{d}_k$ and in CG method we use so called 'conjugate' directions.

5.1.2 Conjugacy

Definition 5.1 ($\mathbf{A}$-conjugate directions)**.** Nonzero directions $\mathbf{d}_0, \dots, \mathbf{d}_{m-1}$ are called $\mathbf{A}$-*conjugate* if

$$\mathbf{d}_i^\top \mathbf{A}\mathbf{d}_j = 0 \qquad \forall i \neq j.$$

Since $\mathbf{A} \succ 0$, the bilinear form

$$\langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{A}} := \mathbf{u}^\top \mathbf{A}\mathbf{v}$$

is an inner product. So the geometric interpretation is immediate: $\mathbf{A}$-conjugacy is just orthogonality in the metric induced by $\mathbf{A}$.

Example 5.1 (Eigenvectors give conjugate directions)**.** If $\mathbf{d}_1, \dots, \mathbf{d}_n$ are orthogonal eigenvectors of $\mathbf{A}$, then they are automatically $\mathbf{A}$-conjugate because

$$\mathbf{d}_i^\top \mathbf{A}\mathbf{d}_j = \lambda_j \mathbf{d}_i^\top \mathbf{d}_j = 0 \qquad (i \neq j).$$

For a symmetric matrix, eigenvectors corresponding to distinct eigenvalues are already orthogonal. If an eigenvalue has multiplicity greater than one, then we simply choose an orthogonal basis inside its eigenspace. So one way to obtain conjugate directions is to diagonalize $\mathbf{A}$ and take an orthogonal eigenbasis. The problem, of course, is that storing or computing a full eigenbasis is typically too expensive if our goal was to stay close to gradient descent.

Claim 5.1. Any nonzero $\mathbf{A}$-conjugate family is linearly independent.

Proof: Suppose

$$\sum_{i=0}^{m-1} c_i \mathbf{d}_i = \mathbf{0}.$$

Fix j and multiply by $\mathbf{d}_j^\top \mathbf{A}$:

$$0 = \mathbf{d}_j^\top \mathbf{A} \sum_{i=0}^{m-1} c_i \mathbf{d}_i = \sum_{i=0}^{m-1} c_i \mathbf{d}_j^\top \mathbf{A}\mathbf{d}_i = c_j \mathbf{d}_j^\top \mathbf{A}\mathbf{d}_j.$$

Because $\mathbf{A} \succ 0$ and $\mathbf{d}_j \neq \mathbf{0}$, the factor $\mathbf{d}_j^\top \mathbf{A}\mathbf{d}_j$ is positive, hence $c_j = 0$. This holds for every j. □

5.1.3 Finite termination

Now suppose $\mathbf{d}_0, \ldots, \mathbf{d}_{n-1}$ is a nonzero $\mathbf{A}$-conjugate basis and let $\mathbf{x}^\star$ denote the minimizer of (5.1). Because the directions form a basis, we can expand

$$\mathbf{x}^\star - \mathbf{x}_0 = \sum_{i=0}^{n-1} \eta_i \mathbf{d}_i.$$

Multiplying by $\mathbf{d}_j^\top \mathbf{A}$ gives

$$\mathbf{d}_j^\top \mathbf{A}(\mathbf{x}^\star - \mathbf{x}_0) = \eta_j \mathbf{d}_j^\top \mathbf{A}\mathbf{d}_j.$$

Since $\mathbf{A}\mathbf{x}^\star = \mathbf{b}$, we obtain

$$\eta_j = \frac{\mathbf{d}_j^\top (\mathbf{b} - \mathbf{A}\mathbf{x}_0)}{\mathbf{d}_j^\top \mathbf{A}\mathbf{d}_j} = -\frac{\mathbf{d}_j^\top \mathbf{g}_0}{\mathbf{d}_j^\top \mathbf{A}\mathbf{d}_j}.$$

Claim 5.2. For every $k \geq 0$,

$$\mathbf{g}_k^\top \mathbf{d}_k = \mathbf{g}_0^\top \mathbf{d}_k.$$

Proof: Using

$$\mathbf{x}_k = \mathbf{x}_0 + \sum_{i=0}^{k-1} \alpha_i \mathbf{d}_i,$$

we obtain

$$\begin{aligned}
\mathbf{g}_k^\top \mathbf{d}_k &= (\mathbf{A}\mathbf{x}_k - \mathbf{b})^\top \mathbf{d}_k \\
&= \Big(\mathbf{A}\mathbf{x}_0 - \mathbf{b} + \sum_{i=0}^{k-1} \alpha_i \mathbf{A}\mathbf{d}_i\Big)^\top \mathbf{d}_k \\
&= \mathbf{g}_0^\top \mathbf{d}_k + \sum_{i=0}^{k-1} \alpha_i \mathbf{d}_i^\top \mathbf{A}\mathbf{d}_k.
\end{aligned}$$

All terms in the sum vanish because $i < k$ and the directions are $\mathbf{A}$-conjugate. □

So the coefficient above may also be written as

$$\eta_j = -\frac{\mathbf{d}_j^\top \mathbf{g}_j}{\mathbf{d}_j^\top \mathbf{A}\mathbf{d}_j},$$

which is exactly the line-search coefficient from (5.2) at step j. Therefore, if we move once along each conjugate direction with the corresponding exact coefficient, we must land at the minimizer after at most n steps.

Claim 5.3. Let

$$\mathbf{x}_k = \mathbf{x}_0 + \sum_{i=0}^{k-1} \alpha_i \mathbf{d}_i, \qquad \alpha_i = -\frac{\mathbf{g}_i^\top \mathbf{d}_i}{\mathbf{d}_i^\top \mathbf{A} \mathbf{d}_i}.$$

Then

$$\mathbf{g}_k^\top \mathbf{d}_j = 0 \qquad \forall j < k.$$

Proof: For each $j < k$,

$$\begin{aligned}
\mathbf{g}_k^\top \mathbf{d}_j &= (\mathbf{A}\mathbf{x}_k - \mathbf{b})^\top \mathbf{d}_j \\
&= \Big(\mathbf{A}\mathbf{x}_0 - \mathbf{b} + \sum_{i=0}^{k-1} \alpha_i \mathbf{A}\mathbf{d}_i\Big)^\top \mathbf{d}_j \\
&= \mathbf{g}_0^\top \mathbf{d}_j + \alpha_j \mathbf{d}_j^\top \mathbf{A}\mathbf{d}_j,
\end{aligned}$$

because all terms with $i \neq j$ vanish by conjugacy. By the formula for α_j, the right-hand side is zero. □

Theorem 5.1 (Method of conjugate directions)**.** For every $k \geq 1$, the iterate $\mathbf{x}_k$ is the unique minimizer of f over the affine space

$$\mathbf{x}_0 + \operatorname{span}\{\mathbf{d}_0, \ldots, \mathbf{d}_{k-1}\}.$$

In particular, $\mathbf{x}_n = \mathbf{x}^\star$.

Proof: Every point in the affine space can be written as

$$\mathbf{x} = \mathbf{x}_k + \sum_{j=0}^{k-1} c_j \mathbf{d}_j.$$

The directional derivative of f at $\mathbf{x}_k$ along any tangent vector $\sum_{j=0}^{k-1} c_j \mathbf{d}_j$ is

$$\mathbf{g}_k^\top \sum_{j=0}^{k-1} c_j \mathbf{d}_j = \sum_{j=0}^{k-1} c_j \mathbf{g}_k^\top \mathbf{d}_j = 0$$

by Claim 5.3. Since the restriction of (5.1) to any affine subspace is strictly convex, this stationary point is the unique minimizer on that subspace. For $k = n$ the subspace is the whole $\mathbb{R}^n$ because the conjugate directions form a basis, so $\mathbf{x}_n = \mathbf{x}^\star$. □

Claim 5.4 (Partial optimality)**.** Fix $k \in \{0, \ldots, n-1\}$ and let

$$\mathbf{x} = \mathbf{x}_k + \sum_{i=k}^{n-1} \delta_i \mathbf{d}_i.$$

Then

$$\nabla f(\mathbf{x})^\top \mathbf{d}_j = 0 \qquad \forall j < k.$$

Consequently, $\mathbf{x}$ is the unique minimizer of f over the affine space

$$\mathbf{x} + \operatorname{span}\{\mathbf{d}_0, \dots, \mathbf{d}_{k-1}\}.$$

Proof: Since

$$\nabla f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b} = \mathbf{g}_k + \sum_{i=k}^{n-1} \delta_i \mathbf{A}\mathbf{d}_i,$$

we obtain, for every $j < k$,

$$\nabla f(\mathbf{x})^\top \mathbf{d}_j = \mathbf{g}_k^\top \mathbf{d}_j + \sum_{i=k}^{n-1} \delta_i \mathbf{d}_i^\top \mathbf{A}\mathbf{d}_j.$$

The first term is zero by Claim 5.3, and every term in the sum is zero by conjugacy. Hence the directional derivative of f vanishes along every vector in $\operatorname{span}\{\mathbf{d}_0, \dots, \mathbf{d}_{k-1}\}$. Since the restriction of (5.1) to any affine subspace is strictly convex, $\mathbf{x}$ is the unique minimizer on the stated affine space. □

This is the partial-optimality property: once the directions are conjugate, a new line minimization does not destroy the optimality already achieved along the previous ones (see fig. 5.1)

5.1.4 A coordinate-decoupling interpretation

Let

$$\mathbf{C} = \begin{bmatrix} \mathbf{d}_0 & \mathbf{d}_1 & \cdots & \mathbf{d}_{n-1} \end{bmatrix}.$$

By Claim 5.1, $\mathbf{C}$ is invertible. Introduce the coordinates

$$\hat{\mathbf{x}} := \mathbf{C}^{-1}\mathbf{x}, \qquad \mathbf{x} = \mathbf{C}\hat{\mathbf{x}}.$$

Then

$$\hat{f}(\hat{\mathbf{x}}) := f(\mathbf{C}\hat{\mathbf{x}}) = \frac{1}{2}\hat{\mathbf{x}}^\top(\mathbf{C}^\top\mathbf{A}\mathbf{C})\hat{\mathbf{x}} - \hat{\mathbf{x}}^\top\mathbf{C}^\top\mathbf{b}.$$

Because the columns of $\mathbf{C}$ are $\mathbf{A}$-conjugate, the matrix $\mathbf{C}^\top\mathbf{A}\mathbf{C}$ is diagonal. If we set

$$\mu_i := \mathbf{d}_i^\top \mathbf{A}\mathbf{d}_i, \qquad \beta_i := \mathbf{d}_i^\top \mathbf{b}, \qquad i = 0, \dots, n-1,$$

then

$$\hat{f}(\hat{\mathbf{x}}) = \sum_{i=0}^{n-1} \left(\frac{1}{2}\mu_i \hat{x}_i^2 - \beta_i \hat{x}_i \right).$$

So each coordinate is completely decoupled from the others.

Now let $\mathbf{e}_k$ denote the kth standard basis vector. Since the kth column of $\mathbf{C}$ is $\mathbf{d}_k$, we have

$$\mathbf{d}_k = \mathbf{C}\mathbf{e}_k.$$

Therefore a step along the direction $\mathbf{d}_k$,

$$\mathbf{x}^+ = \mathbf{x} + \alpha \mathbf{d}_k,$$

corresponds exactly to a step along the kth coordinate axis in the transformed variables:

$$\mathbf{x}^+ = \mathbf{C}\hat{\mathbf{x}} + \alpha\mathbf{C}\mathbf{e}_k = \mathbf{C}(\hat{\mathbf{x}} + \alpha\mathbf{e}_k) \quad \Longrightarrow \quad \hat{\mathbf{x}}^+ = \hat{\mathbf{x}} + \alpha\mathbf{e}_k.$$

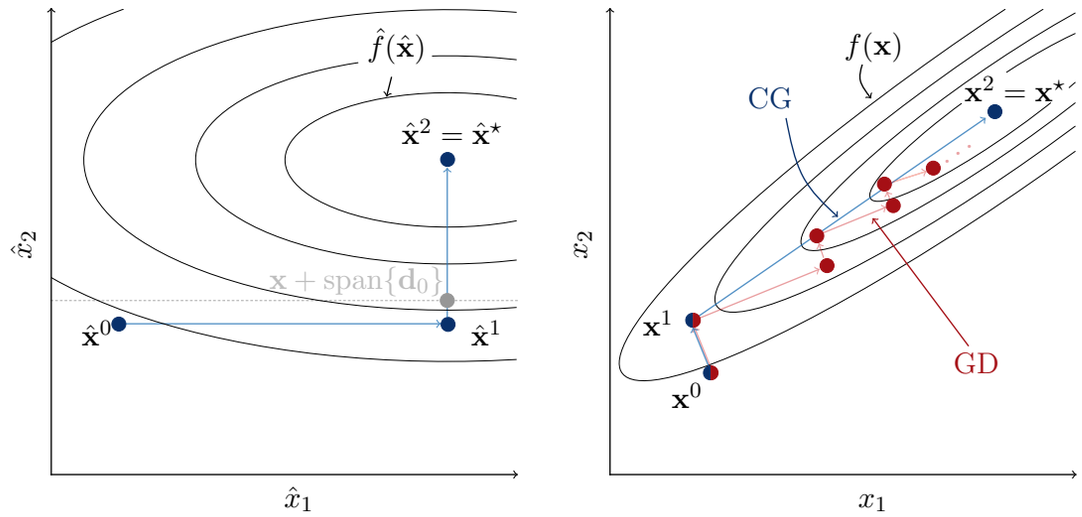

Figure 5.1: A geometric picture of conjugate directions. Left: after the change of variables $\hat{\mathbf{x}} = \mathbf{C}^{-1}\mathbf{x}$, the quadratic form becomes diagonal and each step changes only one coordinate, so the method is exact coordinate descent in the transformed variables. Right: mapping the same iterates back to the original variables produces the conjugate-gradient path on the rotated level sets; the faint red path shows the local steepest-descent zig-zag for comparison. In the left corner of the figure the grey part tries to illustrate partial optimality: dark grey point is an optimal point for dashed affine subspace (see Claim 5.4)

.

Thus, in the transformed coordinates, the method of conjugate directions is exact coordinate descent (see fig. 5.1).

So the next question is unavoidable: how should one choose the conjugate directions? We would like a method whose computational profile stays much closer to gradient descent than to a dense second-order scheme. A direct construction of the full matrix $\mathbf{C}$ defeats that goal, because it requires $O(n^2)$ memory.

One naive idea is to generate random directions and then orthogonalize them by a Gram–Schmidt procedure in the $\mathbf{A}$-inner product

$$\langle \mathbf{u}, \mathbf{v} \rangle_{\mathbf{A}} = \mathbf{u}^\top \mathbf{A} \mathbf{v}.$$

This would indeed produce $\mathbf{A}$-conjugate directions, but it still requires storing the previously generated basis vectors and therefore remains an $O(n^2)$-memory procedure. Moreover, the computational cost will be $O(n^3)$. The key observation is that the partial optimality property proved above (see Claim 5.4) means that we do not need to keep the whole conjugate basis explicitly. This is exactly the point where linear conjugate gradient enters: it generates the needed directions on the fly.

5.2 Linear conjugate gradient

The idea of CG is to construct conjugate directions on the fly, using only the current gradient and the previous search direction:

$$\mathbf{d}_0 = -\mathbf{g}_0, \qquad \mathbf{d}_{k+1} = -\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k. \tag{5.3}$$

We choose β_k so that $\mathbf{d}_{k+1}$ is conjugate to $\mathbf{d}_k$:

$$0 = \mathbf{d}_{k+1}^\top \mathbf{A}\mathbf{d}_k = (-\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k)^\top \mathbf{A}\mathbf{d}_k,$$

hence

$$\beta_k = \frac{\mathbf{g}_{k+1}^\top \mathbf{A}\mathbf{d}_k}{\mathbf{d}_k^\top \mathbf{A}\mathbf{d}_k}. \tag{5.4}$$

At first sight this only enforces conjugacy with the immediately previous direction. For a quadratic with exact line search, however, this is already enough: all the other orthogonality relations propagate automatically.

Definition 5.2 (Krylov subspace)**.** Given $\mathbf{A}$ and $\mathbf{g}_0$, the kth Krylov subspace is

$$\mathcal{K}_k(\mathbf{A}, \mathbf{g}_0) := \operatorname{span}\{\mathbf{g}_0, \mathbf{A}\mathbf{g}_0, \ldots, \mathbf{A}^{k-1}\mathbf{g}_0\}.$$

Theorem 5.2 (Core properties of linear CG)**.** Consider the recurrence (5.3) together with exact line search. Then for every $k \geq 0$:

1. $\mathbf{g}_k, \mathbf{d}_k \in \mathcal{K}_{k+1}(\mathbf{A}, \mathbf{g}_0)$;
2. the gradients are mutually orthogonal:
$$\mathbf{g}_i^\top \mathbf{g}_j = 0 \qquad \forall i \neq j;$$
3. the directions are mutually $\mathbf{A}$-conjugate:
$$\mathbf{d}_i^\top \mathbf{A}\mathbf{d}_j = 0 \qquad \forall i \neq j.$$

Proof: We prove the three statements simultaneously by induction on k.

For $k = 0$, we have

$$\mathbf{g}_0 \in \mathcal{K}_1(\mathbf{A}, \mathbf{g}_0), \qquad \mathbf{d}_0 = -\mathbf{g}_0 \in \mathcal{K}_1(\mathbf{A}, \mathbf{g}_0),$$

and the orthogonality and conjugacy statements are empty.

Assume the three claims hold up to index k. We prove them for index $k+1$.

Step 1: Krylov membership. Since

$$\mathbf{g}_{k+1} = \mathbf{A}\mathbf{x}_{k+1} - \mathbf{b} = \mathbf{A}(\mathbf{x}_k + \alpha_k \mathbf{d}_k) - \mathbf{b} = \mathbf{g}_k + \alpha_k \mathbf{A}\mathbf{d}_k,$$

and $\mathbf{g}_k, \mathbf{d}_k \in \mathcal{K}_{k+1}(\mathbf{A}, \mathbf{g}_0)$, we obtain

$$\mathbf{g}_{k+1} \in \mathcal{K}_{k+2}(\mathbf{A}, \mathbf{g}_0).$$

Then the recurrence (5.3) gives

$$\mathbf{d}_{k+1} = -\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k \in \mathcal{K}_{k+2}(\mathbf{A}, \mathbf{g}_0).$$

This proves item 1 for $k+1$.

Step 2: Orthogonality of the gradients. By the induction hypothesis, for each $j \leq k$ the directions $\mathbf{d}_0, \ldots, \mathbf{d}_j$ are nonzero and mutually $\mathbf{A}$-conjugate, hence linearly independent by Claim 5.1. By Step 1 they belong to $\mathcal{K}_{j+1}(\mathbf{A}, \mathbf{g}_0)$, and this Krylov space is spanned by $j+1$ vectors. Therefore $\mathbf{d}_0, \ldots, \mathbf{d}_j$ form a basis of $\mathcal{K}_{j+1}(\mathbf{A}, \mathbf{g}_0)$.

Now fix $j \leq k$. Since $\mathbf{g}_j \in \mathcal{K}_{j+1}(\mathbf{A}, \mathbf{g}_0)$, we can write

$$\mathbf{g}_j = \sum_{i=0}^{j} \xi_i \mathbf{d}_i.$$

Moreover, since $\mathbf{d}_0, \ldots, \mathbf{d}_k$ are conjugate, the directional-orthogonality result Claim 5.3 applied up to step $k+1$ gives

$$\mathbf{g}_{k+1}^\top \mathbf{d}_i = 0 \qquad \forall i \leq k.$$

Therefore

$$\mathbf{g}_{k+1}^\top \mathbf{g}_j = \sum_{i=0}^{j} \xi_i \mathbf{g}_{k+1}^\top \mathbf{d}_i = 0 \qquad \forall j \leq k.$$

So $\mathbf{g}_{k+1}$ is orthogonal to all previous gradients, and item 2 follows.

Step 3: Conjugacy of the directions. For $j = k$, the choice of β_k in (5.4) gives

$$\mathbf{d}_{k+1}^\top \mathbf{A} \mathbf{d}_k = 0.$$

Now take $j < k$. From

$$\mathbf{g}_{j+1} = \mathbf{A}\mathbf{x}_{j+1} - \mathbf{b} = \mathbf{A}(\mathbf{x}_j + \alpha_j \mathbf{d}_j) - \mathbf{b} = \mathbf{g}_j + \alpha_j \mathbf{A}\mathbf{d}_j$$

we get

$$\mathbf{A}\mathbf{d}_j = \frac{\mathbf{g}_{j+1} - \mathbf{g}_j}{\alpha_j}.$$

Hence

$$\begin{aligned}
\mathbf{d}_{k+1}^\top \mathbf{A} \mathbf{d}_j &= (-\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k)^\top \mathbf{A}\mathbf{d}_j \\
&= -\mathbf{g}_{k+1}^\top \mathbf{A}\mathbf{d}_j + \beta_k \mathbf{d}_k^\top \mathbf{A}\mathbf{d}_j \\
&= -\frac{\mathbf{g}_{k+1}^\top (\mathbf{g}_{j+1} - \mathbf{g}_j)}{\alpha_j} + 0 \\
&= 0,
\end{aligned}$$

because $\mathbf{d}_k^\top \mathbf{A}\mathbf{d}_j = 0$ by the induction hypothesis and $\mathbf{g}_{k+1}^\top \mathbf{g}_{j+1} = \mathbf{g}_{k+1}^\top \mathbf{g}_j = 0$ by Step 2. Thus $\mathbf{d}_{k+1}$ is conjugate to every previous direction, proving item 3. □

The previous section tells us what conjugate directions should achieve. This theorem explains why the short recurrence achieves it: CG keeps growing a Krylov space and reconstructs the needed conjugacy without storing a full basis.

Claim 5.5 (Practical coefficients)**.** For linear CG,

$$\alpha_k = \frac{\mathbf{g}_k^\top \mathbf{g}_k}{\mathbf{d}_k^\top \mathbf{A}\mathbf{d}_k}, \qquad \beta_k = \frac{\mathbf{g}_{k+1}^\top \mathbf{g}_{k+1}}{\mathbf{g}_k^\top \mathbf{g}_k}. \tag{5.5}$$

Proof: From $\mathbf{d}_k = -\mathbf{g}_k + \beta_{k-1}\mathbf{d}_{k-1}$ and $\mathbf{g}_k^\top \mathbf{d}_{k-1} = 0$, we get

$$\mathbf{g}_k^\top \mathbf{d}_k = -\mathbf{g}_k^\top \mathbf{g}_k.$$

Substituting this into (5.2) yields the formula for α_k.

Next,

$$\mathbf{g}_{k+1} = \mathbf{g}_k + \alpha_k \mathbf{A}\mathbf{d}_k \quad \Longrightarrow \quad \mathbf{A}\mathbf{d}_k = \frac{\mathbf{g}_{k+1} - \mathbf{g}_k}{\alpha_k}.$$

Insert this identity into (5.4) and use the orthogonality $\mathbf{g}_{k+1}^\top \mathbf{g}_k = 0$:

$$\beta_k = \frac{\mathbf{g}_{k+1}^\top (\mathbf{g}_{k+1} - \mathbf{g}_k)/\alpha_k}{\mathbf{d}_k^\top \mathbf{A}\mathbf{d}_k} = \frac{\mathbf{g}_{k+1}^\top \mathbf{g}_{k+1}}{\mathbf{g}_k^\top \mathbf{g}_k}.$$

□

Algorithm 5.1 Linear Conjugate Gradient

Require: SPD matrix $\mathbf{A}$, right-hand side $\mathbf{b}$, initial guess $\mathbf{x}_0$, tolerance $\varepsilon > 0$

```
g_0 ← A x_0 − b,  d_0 ← −g_0,  k ← 0
while ‖g_k‖ > ε max{1, ‖b‖} do
    α_k ← (g_k^⊤ g_k) / (d_k^⊤ A d_k)
    x_{k+1} ← x_k + α_k d_k
    g_{k+1} ← g_k + α_k A d_k
    β_k ← (g_{k+1}^⊤ g_{k+1}) / (g_k^⊤ g_k)
    d_{k+1} ← −g_{k+1} + β_k d_k
    k ← k + 1
end while
return x_k
```

Important notes on CG.

- In exact arithmetic, CG terminates in at most n iterations. On a real computer, however, the directions are only approximately conjugate, so in practice one stops when the gradient or residual becomes sufficiently small.

- For dense matrices, one iteration costs $O(n^2)$, since the dominant operation is the matrix-vector product $\mathbf{A}\mathbf{d}_k$. Therefore, n iterations cost $O(n^3)$. The additional working memory beyond storing $\mathbf{A}$ is only $O(n)$, although the total memory usage is still $O(n^2)$ because of the matrix itself. At first sight this may make the method look unappealing; however, the next points explain why CG is often very advantageous.

- If $\mathbf{A}$ is sparse, then the cost of computing $\mathbf{Av}$ can be much smaller than $O(n^2)$ and is typically proportional to the number of nonzero entries. This is precisely the regime where iterative methods become much more attractive than dense factorizations. Matrices arising from bag-of-words models in NLP are a typical example of this kind of structure.

- In contrast to direct factorizations, CG comes with a tolerance parameter ε, so one can stop long before n iterations and still obtain a very accurate approximation. Moreover, after k steps the iterate is optimal over the affine subspace generated by the directions constructed so far.

- If the rows of $\mathbf{A}$ are distributed across several processes, then block-row matrix-vector products can be computed in parallel. Since the matrix remains fixed throughout the optimization process, it does not need to be redistributed at every iteration. This matters in practice because data movement can be expensive, and memory bandwidth is often much slower than the floating-point additions and multiplications themselves (see fig. 5.2).

- Finally, if $\mathbf{A}$ is a Hessian, one may only need Hessian-vector products rather than forming the entire Hessian explicitly.

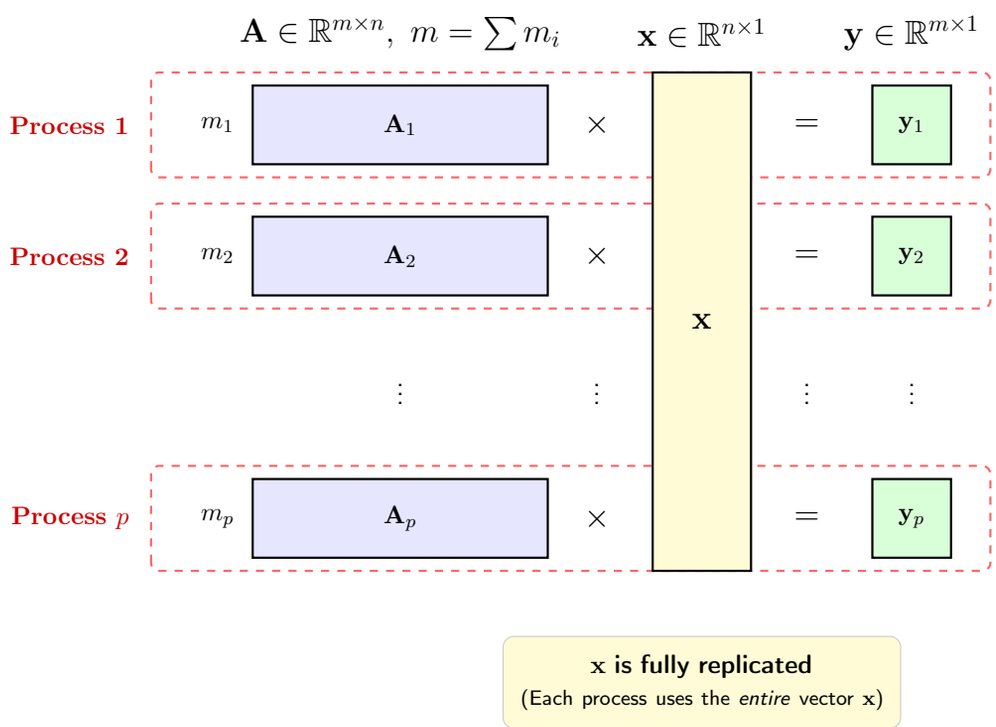

Figure 5.2: Distributed block-row matrix-vector product. Matrix $\mathbf{A}$ and result vector $\mathbf{y}$ are partitioned across p processes. The entire vector $\mathbf{x}$ is required at each node, represented by the single unbroken block spanning all processes. This architecture minimizes decreases the computations and memory costs per node by p.

5.3 Convergence speed

The convergence discussion is driven by the spectrum of $\mathbf{A}$.

Theorem 5.3. The following spectral facts hold for linear CG.

1. If $\mathbf{A}$ has only r distinct eigenvalues, then CG converges in at most r iterations.
2. If the eigenvalues form a few tight clusters, then in practice CG often converges in about the number of clusters rather than in the full dimension.
3. If $\mu = \lambda_{\min}(\mathbf{A})$ and $L = \lambda_{\max}(\mathbf{A})$, then the quadratic (5.1) is μ-strongly convex and L-smooth, and the classical CG estimate is

$$f(\mathbf{x}_k) - f(\mathbf{x}^\star) \le 4 \left(\frac{\sqrt{L/\mu} - 1}{\sqrt{L/\mu} + 1} \right)^{2k} \big(f(\mathbf{x}_0) - f(\mathbf{x}^\star) \big).$$

The third item is the clean comparison with Chapter 2. Gradient descent has a linear rate with factor $1 - \mu/L$; see theorem 2.8. CG replaces the dependence on $\kappa = L/\mu$ by a much milder dependence on $\sqrt{\kappa}$. Also, the estimate is quite pessimistic and not very accurate, so in practice algorithm may be faster.

Example 5.2 (Conditioning experiment)**.** Take

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x}, \qquad \mathbf{A} = \mathrm{Diag}(a_1, \dots, a_n), \qquad \mathbf{b} \sim \mathcal{N}(0, \mathbf{I}_n),$$

with

$$a_i = \begin{cases} 1, & i = 1, \\ \xi_i, \ \xi_i \sim \mathrm{Unif}(1, \kappa), & 2 \le i \le n-1, \\ \kappa, & i = n. \end{cases}$$

Compare the iteration counts of gradient descent and CG as the condition number κ increases (see fig. 5.3).

5.4 Preconditioning

The next practical question is obvious: what if the matrix is SPD but still badly conditioned? The answer is to change variables so that CG sees a more favorable matrix.

Let $\mathbf{S}$ be invertible and define

$$\tilde{\mathbf{x}} := \mathbf{S}\mathbf{x}.$$

Then

$$\mathbf{A}\mathbf{x} = \mathbf{b} \quad \Longleftrightarrow \quad \underbrace{\mathbf{S}^{-\top}\mathbf{A}\mathbf{S}^{-1}}_{=:\tilde{\mathbf{A}} \succ 0} \tilde{\mathbf{x}} = \underbrace{\mathbf{S}^{-\top}\mathbf{b}}_{=:\tilde{\mathbf{b}}}.$$

If $\mathbf{S}$ is chosen so that $\tilde{\mathbf{A}}$ is much better conditioned than $\mathbf{A}$, then CG on the transformed problem should converge much faster.

The clean way to derive the method is to apply the ordinary linear CG scheme from section 5.2 to the transformed system and then map every quantity back to the original variables.

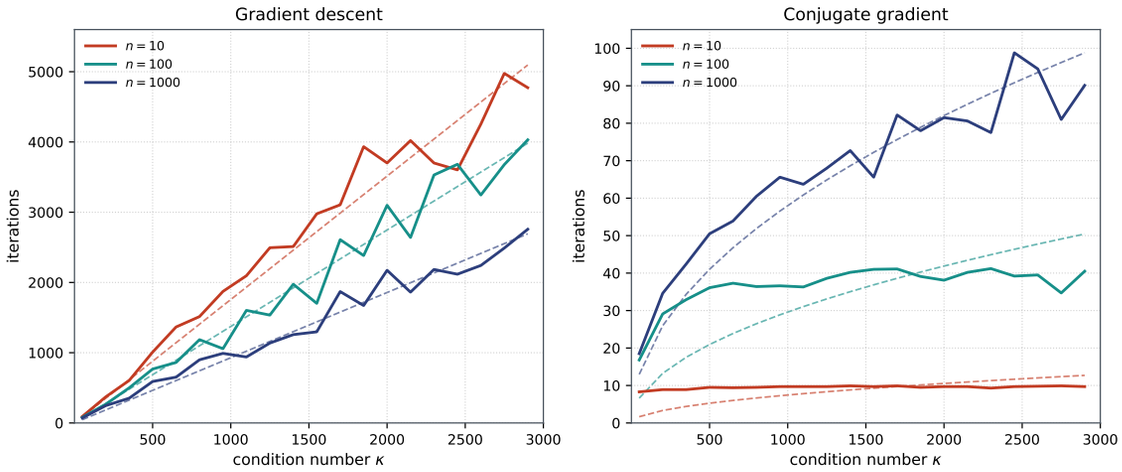

Figure 5.3: A conditioning experiment for gradient descent and conjugate gradient. Left: on this random diagonal ensemble, the main visible effect for gradient descent is the roughly linear growth with the condition number; in this test the curves for larger n happen to lie lower, so the point of the panel is the κ-dependence rather than a monotone dependence on n. Right: for CG the growth is much slower and follows the familiar $O(\sqrt{\kappa})$ pattern until the finite-termination ceiling at n becomes visible, most clearly for $n = 10$. The dashed lines show the corresponding κ- and $\sqrt{\kappa}$-type approximations for each solid curve.

Claim 5.6 (PCG as linear CG in transformed coordinates)**.** Let

$$\tilde{\mathbf{g}}_k := \tilde{\mathbf{A}}\tilde{\mathbf{x}}_k - \tilde{\mathbf{b}}, \qquad \tilde{\mathbf{d}}_0 = -\tilde{\mathbf{g}}_0, \qquad \tilde{\mathbf{d}}_{k+1} = -\tilde{\mathbf{g}}_{k+1} + \tilde{\beta}_k \tilde{\mathbf{d}}_k$$

be the ordinary CG recurrence for the transformed system

$$\tilde{\mathbf{A}}\tilde{\mathbf{x}} = \tilde{\mathbf{b}}.$$

Define

$$\mathbf{x}_k := \mathbf{S}^{-1}\tilde{\mathbf{x}}_k.$$

Since the transformed iterate update is

$$\tilde{\mathbf{x}}_{k+1} = \tilde{\mathbf{x}}_k + \tilde{\alpha}_k \tilde{\mathbf{d}}_k,$$

the corresponding search directions in the original variables must be pulled back as

$$\mathbf{d}_k := \mathbf{S}^{-1}\tilde{\mathbf{d}}_k,$$

because then

$$\mathbf{x}_{k+1} = \mathbf{S}^{-1}\tilde{\mathbf{x}}_{k+1} = \mathbf{S}^{-1}(\tilde{\mathbf{x}}_k + \tilde{\alpha}_k \tilde{\mathbf{d}}_k) = \mathbf{x}_k + \tilde{\alpha}_k \mathbf{d}_k.$$

Finally define the residuals of the original system by

$$\mathbf{g}_k := \mathbf{A}\mathbf{x}_k - \mathbf{b}.$$

Set

$$\alpha_k := \tilde{\alpha}_k, \qquad \beta_k := \tilde{\beta}_k.$$

Then

$$\tilde{\mathbf{g}}_k = \mathbf{S}^{-\top}\mathbf{g}_k, \qquad \mathbf{d}_0 = -\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_0, \qquad \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k\mathbf{d}_k, \qquad \mathbf{g}_{k+1} = \mathbf{g}_k + \alpha_k\mathbf{A}\mathbf{d}_k,$$

and

$$\mathbf{d}_{k+1} = -\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_{k+1} + \beta_k\mathbf{d}_k.$$

Moreover, the transformed CG coefficients become

$$\alpha_k = \tilde{\alpha}_k = \frac{\mathbf{g}_k^\top\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_k}{\mathbf{d}_k^\top\mathbf{A}\mathbf{d}_k}, \qquad \beta_k = \tilde{\beta}_k = \frac{\mathbf{g}_{k+1}^\top\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_{k+1}}{\mathbf{g}_k^\top\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_k}.$$

Proof: The update formula for $\mathbf{x}_{k+1}$ was already built into the definition of $\mathbf{d}_k$, so it remains to identify how the gradients transform and then substitute that relation into the CG recurrence.
First,

$$\tilde{\mathbf{g}}_k = \tilde{\mathbf{A}}\tilde{\mathbf{x}}_k - \tilde{\mathbf{b}} = \mathbf{S}^{-\top}\mathbf{A}\mathbf{S}^{-1}\tilde{\mathbf{x}}_k - \mathbf{S}^{-\top}\mathbf{b} = \mathbf{S}^{-\top}(\mathbf{A}\mathbf{x}_k - \mathbf{b}) = \mathbf{S}^{-\top}\mathbf{g}_k.$$

Therefore

$$\mathbf{d}_0 = \mathbf{S}^{-1}\tilde{\mathbf{d}}_0 = -\mathbf{S}^{-1}\tilde{\mathbf{g}}_0 = -\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_0.$$

Substituting this into the definition of $\mathbf{g}_{k+1}$ gives

$$\mathbf{g}_{k+1} = \mathbf{A}\mathbf{x}_{k+1} - \mathbf{b} = \mathbf{A}(\mathbf{x}_k + \tilde{\alpha}_k\mathbf{d}_k) - \mathbf{b} = \mathbf{g}_k + \tilde{\alpha}_k\mathbf{A}\mathbf{d}_k.$$

Finally,

$$\mathbf{d}_{k+1} = \mathbf{S}^{-1}\tilde{\mathbf{d}}_{k+1} = -\mathbf{S}^{-1}\tilde{\mathbf{g}}_{k+1} + \tilde{\beta}_k\mathbf{S}^{-1}\tilde{\mathbf{d}}_k = -\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_{k+1} + \tilde{\beta}_k\mathbf{d}_k.$$

For the coefficients, the transformed CG formulas give

$$\tilde{\alpha}_k = \frac{\tilde{\mathbf{g}}_k^\top\tilde{\mathbf{g}}_k}{\tilde{\mathbf{d}}_k^\top\tilde{\mathbf{A}}\tilde{\mathbf{d}}_k}.$$

The numerator satisfies

$$\tilde{\mathbf{g}}_k^\top\tilde{\mathbf{g}}_k = \mathbf{g}_k^\top\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_k,$$

while the denominator becomes

$$\tilde{\mathbf{d}}_k^\top\tilde{\mathbf{A}}\tilde{\mathbf{d}}_k = (\mathbf{S}\mathbf{d}_k)^\top(\mathbf{S}^{-\top}\mathbf{A}\mathbf{S}^{-1})(\mathbf{S}\mathbf{d}_k) = \mathbf{d}_k^\top\mathbf{A}\mathbf{d}_k.$$

Hence

$$\tilde{\alpha}_k = \frac{\mathbf{g}_k^\top\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_k}{\mathbf{d}_k^\top\mathbf{A}\mathbf{d}_k}.$$

Likewise,

$$\tilde{\beta}_k = \frac{\tilde{\mathbf{g}}_{k+1}^\top\tilde{\mathbf{g}}_{k+1}}{\tilde{\mathbf{g}}_k^\top\tilde{\mathbf{g}}_k} = \frac{\mathbf{g}_{k+1}^\top\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_{k+1}}{\mathbf{g}_k^\top\mathbf{S}^{-1}\mathbf{S}^{-\top}\mathbf{g}_k}.$$

Renaming $\tilde{\alpha}_k, \tilde{\beta}_k$ as α_k, β_k in the original coordinates gives the stated recurrence and coefficient formulas. $\square$

At this point the matrix

$$\mathbf{S}^{-1}\mathbf{S}^{-\top}$$

appears everywhere. It is therefore natural to encode the change of variables through the SPD matrix

$$\mathbf{M} := \mathbf{S}^\top\mathbf{S}, \qquad \mathbf{M}^{-1} = \mathbf{S}^{-1}\mathbf{S}^{-\top}.$$

This matrix is the *preconditioner*. The ideal case would be $\mathbf{M} = \mathbf{S}^\top\mathbf{S} = \mathbf{A}$, because then

$$\tilde{\mathbf{A}} = \mathbf{S}^{-\top}\mathbf{A}\mathbf{S}^{-1} = \mathbf{I}.$$

In that ideal situation, CG would solve the transformed problem in one step in exact arithmetic. Of course, such a preconditioner is usually as hard to build as solving the original system, so in practice one looks for a matrix that only approximates $\mathbf{A}$ but is cheap to invert or apply (e.g. diagonal).

Final prseudocode for preconditioned CG can be viewed in algorithm 5.2. The terms colored in crimson are precisely the replacements that turn algorithm 5.1 into its preconditioned version.

Algorithm 5.2 Linear CG and preconditioned CG. In the right column, the terms colored in crimson are the only changes relative to the linear method.

LINEAR CG	PRECONDITIONED CG
Require: SPD matrix $\mathbf{A}$, vector $\mathbf{b}$, initial guess $\mathbf{x}_0$, tolerance $\varepsilon > 0$	**Require:** SPD matrix $\mathbf{A}$, vector $\mathbf{b}$, SPD preconditioner $\mathbf{M}$, initial guess $\mathbf{x}_0$, tolerance $\varepsilon > 0$
$\mathbf{g}_0 \leftarrow \mathbf{A}\mathbf{x}_0 - \mathbf{b}, \;\; \mathbf{d}_0 \leftarrow -\mathbf{g}_0, \;\; k \leftarrow 0$	$\mathbf{g}_0 \leftarrow \mathbf{A}\mathbf{x}_0 - \mathbf{b}, \;\; \mathbf{d}_0 \leftarrow -\mathbf{M}^{-1}\mathbf{g}_0, \;\; k \leftarrow 0$
while $\lVert\mathbf{g}_k\rVert > \varepsilon \max\{1, \lVert\mathbf{b}\rVert\}$ **do**	**while** $\lVert\mathbf{g}_k\rVert > \varepsilon \max\{1, \lVert\mathbf{b}\rVert\}$ **do**
$\alpha_k \leftarrow \dfrac{\mathbf{g}_k^\top\mathbf{g}_k}{\mathbf{d}_k^\top\mathbf{A}\mathbf{d}_k}$	$\alpha_k \leftarrow \dfrac{\mathbf{g}_k^\top\mathbf{M}^{-1}\mathbf{g}_k}{\mathbf{d}_k^\top\mathbf{A}\mathbf{d}_k}$
$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k\mathbf{d}_k$	$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k\mathbf{d}_k$
$\mathbf{g}_{k+1} \leftarrow \mathbf{g}_k + \alpha_k\mathbf{A}\mathbf{d}_k$	$\mathbf{g}_{k+1} \leftarrow \mathbf{g}_k + \alpha_k\mathbf{A}\mathbf{d}_k$
$\beta_k \leftarrow \dfrac{\mathbf{g}_{k+1}^\top\mathbf{g}_{k+1}}{\mathbf{g}_k^\top\mathbf{g}_k}$	$\beta_k \leftarrow \dfrac{\mathbf{g}_{k+1}^\top\mathbf{M}^{-1}\mathbf{g}_{k+1}}{\mathbf{g}_k^\top\mathbf{M}^{-1}\mathbf{g}_k}$
$\mathbf{d}_{k+1} \leftarrow -\mathbf{g}_{k+1} + \beta_k\mathbf{d}_k$	$\mathbf{d}_{k+1} \leftarrow -\mathbf{M}^{-1}\mathbf{g}_{k+1} + \beta_k\mathbf{d}_k$
$k \leftarrow k + 1$	$k \leftarrow k + 1$
end while	**end while**
return $\mathbf{x}_k$	**return** $\mathbf{x}_k$

Example 5.3 (Diagonal preconditioning (see fig. 5.4)). Consider

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \qquad n = 500, \qquad \mathbf{b} = (1, \ldots, 1)^\top,$$

with

$$a_{ij} = \begin{cases} 1 + i^{1.2}, & i = j, \\ 1, & |i-j| = 1 \text{ or } |i-j| = 100, \\ 0, & \text{otherwise}, \end{cases} \qquad \mathbf{M} = \mathrm{Diag}(\mathbf{A}) \approx \mathbf{A}.$$

This is the simplest diagonal preconditioner.

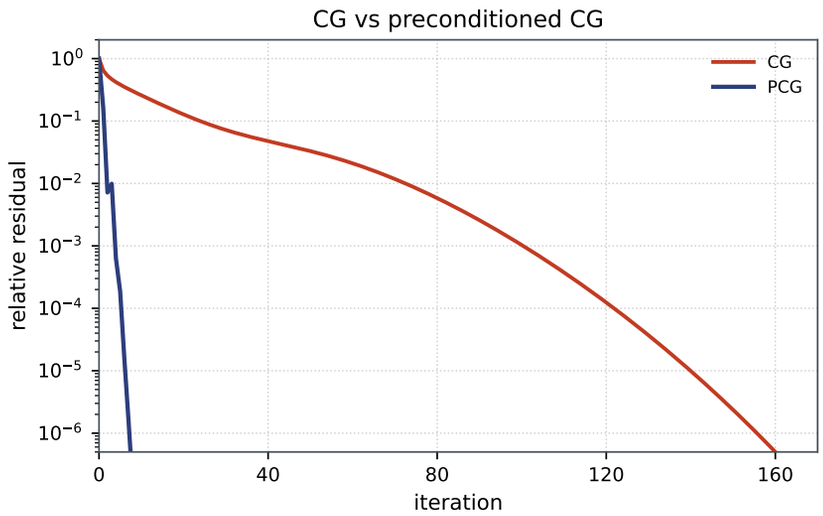

Figure 5.4: A preconditioning example. Even the basic diagonal preconditioner $\mathbf{M} = \mathrm{Diag}(\mathbf{A})$ reduces the residual much faster than plain CG on the same structured SPD system.

This section closes with the right warning: a good preconditioner can change the behavior of CG completely, but finding such a preconditioner is its own modeling problem. Typical first options are diagonal scaling and, for sparse matrices, incomplete Cholesky factorizations that try to preserve the sparsity pattern.

5.5 Nonlinear conjugate gradient

Up to now the quadratic structure has been used everywhere. We now ask what survives for a general smooth objective $f \in C^1(\mathbb{R}^n)$. The answer is that the short recurrence survives, but finite termination disappears and line search becomes an essential part of the method.

5.5.1 Fletcher–Reeves

Let $\mathbf{g}_k := \nabla f(\mathbf{x}_k)$. The Fletcher–Reeves version of nonlinear CG keeps the same update pattern as linear CG:

$$\mathbf{d}_0 = -\mathbf{g}_0, \qquad \mathbf{d}_{k+1} = -\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k, \qquad \beta_k^{\mathrm{FR}} = \frac{\mathbf{g}_{k+1}^\top \mathbf{g}_{k+1}}{\mathbf{g}_k^\top \mathbf{g}_k}.$$

The difference is that the step size is no longer given by (5.5). One now chooses α_k by a line search, for example using the Armijo–Wolfe conditions from Chapter 2. Algorithm 5.3 places linear CG and Fletcher–Reeves side by side. The terms colored in crimson are exactly the parts that change when we pass from the quadratic model to a general smooth objective.

For general objectives, Fletcher–Reeves does *not* automatically guarantee that every $\mathbf{d}_k$ is a descent direction. A standard safeguard is to require the strong Wolfe conditions with $c_2 < 1/2$; then the generated direction is still a descent direction.

Algorithm 5.3 Linear CG and Fletcher–Reeves nonlinear CG. In the right column, the terms colored in crimson are the only changes relative to the linear method.

Linear CG	**Fletcher–Reeves**

Require: SPD matrix $\mathbf{A}$, vector $\mathbf{b}$, initial guess $\mathbf{x}_0$, tolerance $\varepsilon > 0$
$\mathbf{g}_0 \leftarrow \mathbf{A}\mathbf{x}_0 - \mathbf{b},\ \ \mathbf{d}_0 \leftarrow -\mathbf{g}_0,\ \ k \leftarrow 0$
while $\|\mathbf{g}_k\| > \varepsilon \max\{1, \|\mathbf{b}\|\}$ **do**
$\alpha_k \leftarrow \dfrac{\mathbf{g}_k^\top \mathbf{g}_k}{\mathbf{d}_k^\top \mathbf{A}\mathbf{d}_k}$
$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{d}_k$
$\mathbf{g}_{k+1} \leftarrow \mathbf{g}_k + \alpha_k \mathbf{A}\mathbf{d}_k$
$\beta_k \leftarrow \dfrac{\mathbf{g}_{k+1}^\top \mathbf{g}_{k+1}}{\mathbf{g}_k^\top \mathbf{g}_k}$
$\mathbf{d}_{k+1} \leftarrow -\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k$
$k \leftarrow k+1$
end while
return $\mathbf{x}_k$
	Require: f, ∇f, initial point $\mathbf{x}_0$, tolerance $\varepsilon > 0$
	$\mathbf{g}_0 \leftarrow \nabla f(\mathbf{x}_0),\ \ \mathbf{d}_0 \leftarrow -\mathbf{g}_0,\ \ k \leftarrow 0$
	while $\|\mathbf{g}_k\| > \varepsilon \|\nabla f(\mathbf{x}_0)\|$ **do**
	$\alpha_k \leftarrow$ line search
	$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{d}_k$
	$\mathbf{g}_{k+1} \leftarrow \nabla f(\mathbf{x}_{k+1})$
	$\beta_k \leftarrow \dfrac{\mathbf{g}_{k+1}^\top \mathbf{g}_{k+1}}{\mathbf{g}_k^\top \mathbf{g}_k}$
	$\mathbf{d}_{k+1} \leftarrow -\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k$
	$k \leftarrow k+1$
	end while
	return $\mathbf{x}_k$

5.5.2 Other β_k formulas

Different nonlinear-CG schemes differ in how the coefficient β_k is chosen. Let

$$\mathbf{y}_k := \mathbf{g}_{k+1} - \mathbf{g}_k.$$

Common formulas are

$$\beta_k^{\mathrm{PR}} = \frac{\mathbf{g}_{k+1}^\top \mathbf{y}_k}{\mathbf{g}_k^\top \mathbf{g}_k}, \qquad \beta_k^{\mathrm{HS}} = \frac{\mathbf{g}_{k+1}^\top \mathbf{y}_k}{\mathbf{d}_k^\top \mathbf{y}_k},$$
$$\beta_k^{\mathrm{PR+}} = \max\{0, \beta_k^{\mathrm{PR}}\}, \quad \beta_k^{\mathrm{GN}} = \max\{-\beta_k^{\mathrm{FR}}, \min(\beta_k^{\mathrm{PR}}, \beta_k^{\mathrm{FR}})\},$$
$$\beta_k^{\mathrm{DY}} = \frac{\mathbf{g}_{k+1}^\top \mathbf{g}_{k+1}}{\mathbf{d}_k^\top \mathbf{y}_k}, \qquad \beta_k^{\mathrm{HZ}} = \frac{\left\langle \mathbf{g}_{k+1}, \mathbf{y}_k - 2\mathbf{d}_k \frac{\|\mathbf{y}_k\|^2}{\mathbf{d}_k^\top \mathbf{y}_k} \right\rangle}{\mathbf{d}_k^\top \mathbf{y}_k}.$$

Out of this set the most popural option is Polak–Ribiere. On a strictly convex quadratic with exact line search, all of these formulas collapse to the same method. That is why the linear theory remains the right local model for nonlinear CG near a quadratic-like minimizer.

5.5.3 Restarts

For a general nonlinear objective, CG no longer finishes in n steps, and the conjugacy information accumulated in the recurrence can become stale. It is therefore natural to use occasional restarts:

$$\mathbf{d}_k := -\mathbf{g}_k.$$

A standard restart condition is

$$|\mathbf{g}_{k+1}^\top \mathbf{g}_k| > \nu \|\mathbf{g}_{k+1}\|^2, \qquad \nu \approx 0.1. \tag{5.6}$$

The idea is simple and geometric: on a quadratic model, neighboring gradients are orthogonal, so a serious loss of orthogonality is a natural signal to throw away the accumulated direction history and restart from gradient descent.

Example 5.4 (Two standard test problems (see fig. 5.5)). Two standard test problems are:

- the Rosenbrock function

$$f(\mathbf{x}) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2;$$

- an ℓ_2-regularized logistic-type objective

$$f(\mathbf{x}) = \sum_{i=1}^{m} \ln\big(1 + \exp(\langle a_i, \mathbf{x}\rangle)\big) + \frac{\lambda}{2}\|\mathbf{x}\|^2 \to \min_{\mathbf{x}\in\mathbb{R}^n}.$$

These two examples illustrate the practical message of nonlinear conjugate gradient. On Rosenbrock, plain FR can stall while PR or FR with restart behaves much better; on a large smooth problem, nonlinear CG can be substantially more effective than plain gradient descent at the same line-search accuracy.

The memory footprint of nonlinear CG is small and equals $O(n)$. The cost of one iteration is $O(n)$. In problems where gradient descent and nonlinear CG use the same line-search accuracy, nonlinear CG is usually faster in practice because the search direction reuses curvature information from previous steps instead of restarting from the steepest-descent direction every iteration. So, CG and GD have the same memory and computational cost, but CG converges, usually, much faster.

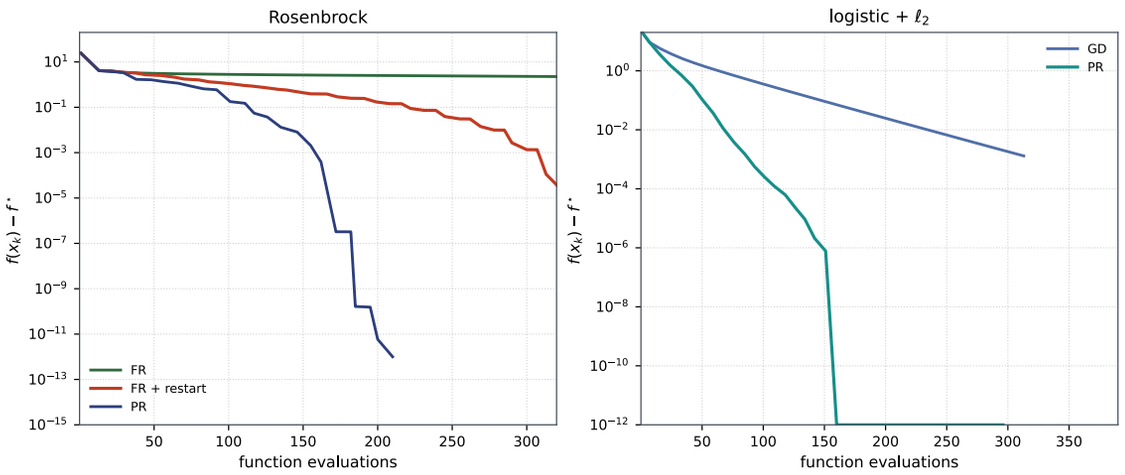

Figure 5.5: Two nonlinear-CG examples. Left: on Rosenbrock, Polak–Ribiere and restarts improve substantially over plain Fletcher–Reeves. Right: on a regularized logistic example, nonlinear CG is much more effective than gradient descent when measured by function evaluations.